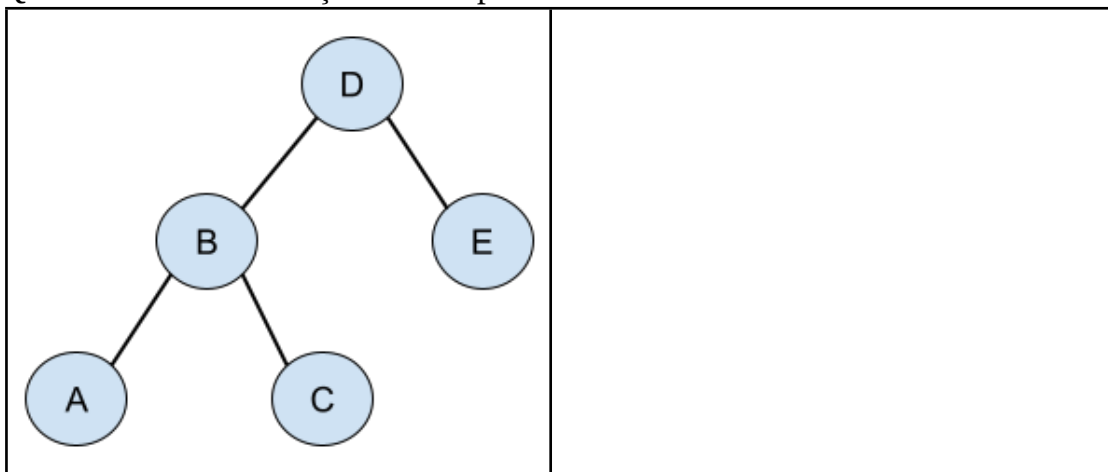


Disciplina: Algoritmos e Estruturas de Dados I (EDI)
Professor: Eduardo de Lucena Falcão
Exercício sobre AVL, Tabelas de Dispersão

AVL

[Implementação de AVL no Replit](#)

- 1) O que é fator de balanceamento, e como ele é calculado?
- 2) O que diferencia uma árvore AVL de uma BST?
- 3) Explique o desempenho de árvores BSTs e AVLs.
- 4) Qual o resultado da rotação do nó D para a direita?



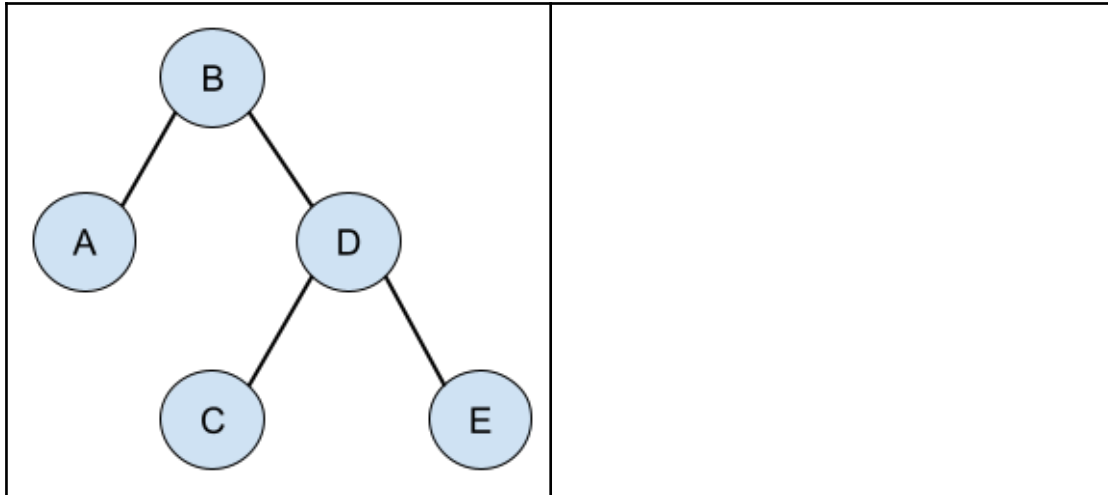
- 5) Escreva o código que rotaciona uma árvore à direita, i.e., que atribui os apontadores das sub-árvores esquerda e sub-árvores direita que precisam de ajuste. (nesse momento não se preocupe com atualização de propriedades como altura e fator de balanço).

```
type BstNode struct {  
    left *BstNode  
    value int  
    height int  
    bf int  
    right *BstNode  
}
```

```
func (root *BstNode) RotRight() *BstNode {
```

}

- 6) Na função que você implementou na questão anterior, por que é fundamental retornar um *BstNode?
- 7) Qual o resultado da rotação do nó B para a esquerda?



- 8) Escreva o código que rotaciona uma árvore à esquerda, i.e., que atribui os apontadores das sub-árvores esquerda e sub-árvores direita que precisam de ajuste. (nesse momento não se preocupe com atualização de propriedades como altura e fator de balanço).

```

func (root *BstNode) RotRight() *BstNode {

}
  
```

- 9) Quando uma árvore AVL é rotacionada, seja para a direita ou para esquerda, algumas propriedades como altura e fator de balanço de alguns nós precisam ser atualizadas. Aponte quais nós, e em qual ordem, precisam atualizar suas propriedades na ilustração que você fez nas questões 4 e 7.
- 10) Implemente a função que atualiza as propriedades (altura e fator de balanço) de um nó.

```

func (root *BstNode) UpdateProperties() *BstNode {
  
```

}

- 11) Reescreva as funções **RotRight** e **RotLeft**, acrescentando as chamadas ao procedimento **UpdateProperties()**.
- 12) Ilustre os 6 casos de desbalanceamento detalhando a altura (h) e fator de balanço (bf) de cada nó: **esquerda-esquerda**, **esquerda-neutro**, **esquerda-direita**, **direita-direita**, **direita-neutro**, **direita-esquerda**.

esquerda-esquerda	esquerda-neutro	esquerda-direita
direita-direita	direita-neutro	direita-esquerda

- 13) Descreva as ações que devem ser realizadas para balancear uma árvore AVL que esteja desbalanceada de acordo com cada um dos seguintes casos: **esquerda-esquerda**, **esquerda-neutro**, **esquerda-direita**, **direita-direita**, **direita-neutro**, **direita-esquerda**.

```
func (root *BstNode) RebalanceLeftLeft() *BstNode {
```

```
return root.RotRight()  
}
```

func (root *BstNode) RebalanceLeftNeutral() *BstNode

func (root *BstNode) RebalanceLeftRight() *BstNode

func (root *BstNode) RebalanceRightRight() *BstNode

func (root *BstNode) RebalanceRightNeutral() *BstNode

func (root *BstNode) RebalanceRightLeft() *BstNode

14) Nas funções que você implementou na questão anterior, por que é fundamental retornar um *BstNode?

15) Reusando as funções escritas na questão 13, e informações de sua resposta da questão 12, implemente a função que rebalanceia uma árvore AVL.

func (root *BstNode) Rebalance() *BstNode

- 16) Considerando a seguinte implementação para adição de um nó em árvores BST, apresente como deveria ser a implementação de adição em árvores AVL.

```
func (root *BstNode) Add(value int){
    if value <= root.value { //insercao à esq
        if root.left == nil {
            root.left = NewNode(value)
        } else {
            root.left.Add(value)
        }
    } else {
        if root.right == nil {
            root.right = NewNode(value)
        } else {
            root.right.Add(value)
        }
    }
}
```

```
func (root *BstNode) Add(value int) *BstNode
```

- 17) Na função implementada na questão anterior, por que é fundamental retornar um *BstNode?

18) Considerando a seguinte implementação para remoção de um nó em árvores BST, apresente como deveria ser a implementação de adição em árvores AVL.

```
func (root *BstNode) Remove(value int) *BstNode {
    if value < root.value {
        root.left = root.left.Remove(value)
    } else if value > root.value {
        root.right = root.right.Remove(value)
    } else { //encontramos o nó a ser removido
        if root.left == nil && root.right == nil { //nó folha
            return nil
        } else if root.left != nil && root.right == nil {
            //caso 2: nó com 1 filho (à esquerda)
            return root.left
        } else if root.left == nil && root.right != nil {
            //caso 2: nó com 1 filho (à direita)
            return root.right
        } else {
            //caso 3: nó com 2 filhos
            maxEsq := root.left.Max()
            root.value = maxEsq
            root.left = root.left.Remove(maxEsq)
            return root
        }
    }
}
```

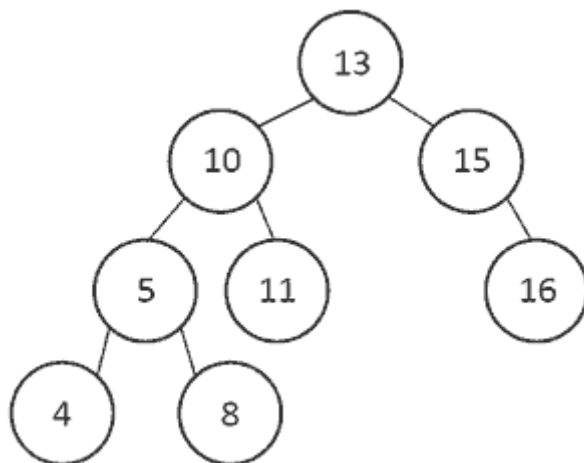
```
func (root *BstNode) Remove(value int) *BstNode {
```

19) Na função implementada na questão anterior, por que é fundamental retornar um *BstNode?

20) Escreva uma função que identifica se uma árvore é AVL.

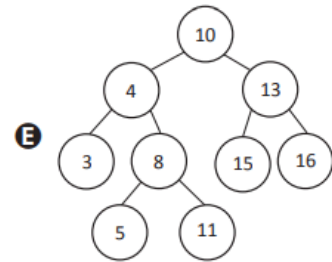
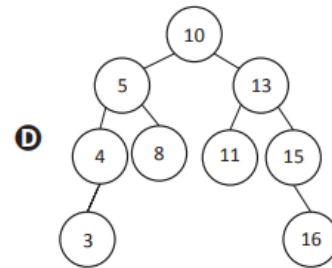
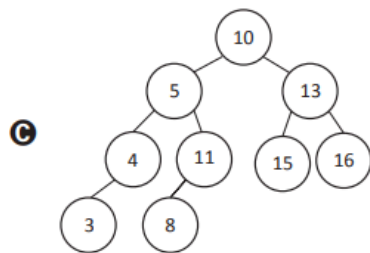
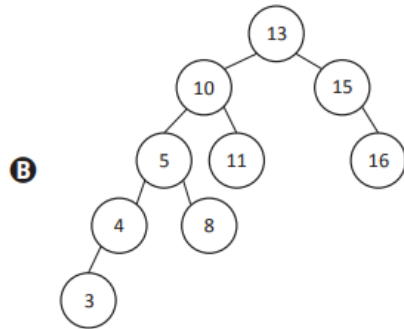
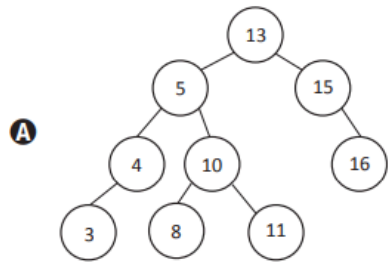
```
func (root *BstNode) IsAVL() bool
```

- 21) Adicione os caracteres de seu nome completo (sem repetição), e detalhe quais operações de rotação / rebalanceamento aconteceriam para que a árvore se mantivesse balanceada.
- 22) Uma árvore AVL é um tipo de árvore binária balanceada na qual a diferença entre as alturas de suas subárvores da esquerda e da direita não pode ser maior do que 1 para qualquer nó. Após a inserção de um nó em uma AVL, a raiz da subárvore de nível mais baixo no qual o novo nó foi inserido é marcada. Se a altura de seus filhos diferir em mais de uma unidade, é realizada uma rotação simples ou uma rotação dupla para igualar suas alturas. A seguir, é apresentado um exemplo de árvore AVL.



Pelo exposto no texto acima, após a inserção de um nó com valor 3 na árvore AVL exemplificada, é correto afirmar que ela ficará com a seguinte configuração:

CENTRO DE TECNOLOGIA
DEPARTAMENTO DE ENGENHARIA DE COMPUTAÇÃO E AUTOMAÇÃO



Tabelas de Dispersão

[Implementação de Tabela Hash no Replit](#)

- 1) Qual é a estrutura de dados básica para implementar uma Tabela de Dispersão (Hash)?

- 2) Como os elementos de diferentes tipos são mapeados para a tabela?

- 3) Mapas e dicionários são TADs que podem ser implementados com Tabelas de Dispersão. Pense nas funções que seriam fundamentais no tipo Map.

```
type Map interface {
```

```
}
```

- 4) Como você criaria a estrutura de dado HashTable?

```
type HashTable struct {
```

```
}
```

- 5) O que são colisões? Quais são as diferentes formas de endereçá-las?

- 6) Escreva o código para as funções que você especificou na questão 3.

Filas de Prioridades (Heaps Binários - [Replit](#))

- 1) Explique o que é uma Fila de Prioridade e o que é uma Heap. Quais são suas aplicações?

- 2) Qual a diferença entre Heap de mínimo e Heap de máximo?

- 3) Pense nas funções que seriam fundamentais no tipo PriorityQueue.

```
type PQ interface {  
  
  
  
  
  
  
  
  
  
}
```

- 4) Como você criaria a estrutura de dado BinaryHeap?

```
type BinaryHeap struct {  
  
  
  
  
  
  
  
  
  
}
```

- 5) Escreva o código para as funções que você especificou na questão anterior.
- 6) Ilustre uma Heap de Mínimo que foi criada através da inserção dos 10 primeiros caracteres do seu nome.

____, ____, ____, ____, ____, ____, ____, ____, ____, ____.

Como as seguintes operações aconteceriam na Heap que você criou?

- Adicionar A
- Adicionar Z
- Poll (desenfileirar o elemento de maior prioridade)
- Remova o elemento correspondente à primeira letra de seu nome

- 7) Ilustre uma Heap de Máximo que foi criada através da inserção dos 10 primeiros caracteres do seu nome.

____, ____, ____, ____, ____, ____, ____, ____, ____, ____.

Como as seguintes operações aconteceriam na Heap que você criou?



CENTRO DE TECNOLOGIA

DEPARTAMENTO DE ENGENHARIA DE COMPUTAÇÃO E AUTOMAÇÃO

- Adicionar A
- Adicionar Z
- Poll (desenfileirar o elemento de maior prioridade)
- Remova o elemento correspondente à primeira letra de seu nome